

Greedy Algorithms

Algorithms and Complexity - Computer Science

1. The greedy choice

A greedy algorithm builds a solution by repeatedly taking the option that looks best right now, never reconsidering. It is the simplest strategy available, and it is correct only when local optimality implies global optimality.

2. When greedy is provably correct

Two properties are needed. The greedy-choice property: some optimal solution begins with the locally best choice. Optimal substructure: an optimal solution contains optimal solutions to its subproblems. Activity selection by earliest finishing time, Huffman coding, and the minimum-spanning-tree algorithms of Kruskal and Prim all satisfy both.

3. Where greedy fails

The 0/1 knapsack problem defeats a greedy choice by value density - taking the densest item first can strand capacity that a different combination would have used. The fractional knapsack, where items may be split, is greedy-solvable. The difference is a reminder to prove the greedy-choice property rather than assume it.

4. Greedy against dynamic programming

Both rely on optimal substructure. Greedy commits to one choice and never revisits it, giving speed; dynamic programming explores the choices it cannot rule out, giving generality at higher cost. Try to prove greedy correct first, and fall back to dynamic programming when the proof fails.